AI-Powered Student Mental Health Assistant

SYSTEM DESIGN & SOFTWARE DOCUMENTATION STUDY NOTES

End-to-End Natural Language Processing, Custom RAG Vector Database,

and Analytical Longitudinal Mental Health Tracking

Author & Software Architect

Md Irfan

MBA (Data Science) | Software Engineer & AI Enthusiast

Academic Scope & Context

MBA Data Science Capstone / Defense Report Notes

Date: June 2026

Deployment: Vercel (Backend API) & GitHub Pages (Frontend UI)

Section 1: Problem Statement

Modern university education exposes students to intense and unique pressures, including academic loads, social adaptation, financial concerns, and career uncertainty. Statistics show that anxiety, chronic stress, and feelings of isolation are highly prevalent in student populations. Despite this growing mental health need, traditional university counseling services face major structural bottlenecks: severe counselor shortages, long wait times (often weeks or months), and high hourly costs. Furthermore, many students hesitate to seek initial help due to social stigma or embarrassment.

This project implements an AI-Powered Student Mental Health Assistant to bridge this resource gap. Designed as a continuous 24/7 empathetic utility, it detects students' emotional states, provides immediate clinically grounded support using cognitive behavioral therapy (CBT) conversational patterns, and recommends verified university coping strategies via semantic Retrieval-Augmented Generation (RAG). By integrating a secure login system and a longitudinal analytics dashboard, the application allows students to privately log daily check-ins, record progress, and track their stress levels alongside academic focus, turning subjective experiences into visual insights.

Section 2: System Requirements

Functional Requirements

1. User Authentication & Security: Users must be able to securely register accounts and sign in. User sessions must be managed via cryptographically signed JWT bearer tokens.

2. Emotion-Aware Streaming Conversations: The core chatbot must analyze user inputs in real-time to categorize the primary emotion (sadness, joy, anger, fear, surprise, or neutral) and stream replies.

3. Empirical Therapy Integration: Conversations must dynamically synthesize evidence-based coping techniques (e.g., Box Breathing, Behavioral Activation) based on the detected emotion.

4. Knowledge Base RAG Lookup: The chatbot must perform semantic queries on a custom vector-style database of academic resources and explicitly cite relevant university materials.

5. Longitudinal Check-Ins: Users must be able to record daily self-assessments rating their stress levels and academic focus on a 1-to-10 scale.

6. Analytical Dashboard Visualization: The application must expose secure analytics endpoints aggregating historical statistics on user emotions, daily chat volume, chatbot feedback, and longitudinal correlation trends.

Non-Functional Requirements

1. Security & Privacy: User passwords must be stored using strong PBKDF2 cryptography with SHA-256 and high iterations (390k) to prevent rainbow table attacks. User sessions must be stateless and secured via HS256 HMAC JWT signatures.

2. Performance & Responsiveness: Chat responses must be delivered using Server-Sent Events (SSE) to achieve sub-second perceived response latency, streaming generated chunks to the client as they are generated.

3. Scalability & Availability: The system must follow a clean separation of concerns (Routers, Services,

Repositories, ML Layer) and support containerization (Docker-Compose) and serverless deployment (Vercel API, PostgreSQL).

4. Usability & UX Design: The web interface must present a modern, glassmorphic UI layout with dark/light mode toggling, inline field validations, and elegant markdown rendering for easy student reading.

Section 3: API Interface Design

The backend implements a clean REST API using FastAPI. It separates authentication, conversation mechanics, and data aggregation. JWT tokens must be supplied in the HTTP Authorization header (e.g., 'Bearer <token>') for all protected routes.

REST API Endpoints

Method	Endpoint Route	Auth Needed	Purpose & Payload Details
POST	/auth/signup	No	Registers a new student user. Requires username, email, password. Returns signed JWT token.
POST	/auth/login	No	Authenticates user. Accepts username/email and password. Returns signed JWT access token.
GET	/auth/me	Yes	Retrieves authenticated profile metadata (user ID, username, email).
POST	/chat	Yes	Primary chatbot endpoint. Accepts message body. Streams Server-Sent Events (SSE) chat chunks.
POST	/chat/feedback/{id}	Yes	Records feedback score (1 to 5 rating scale) for a given chat response ID to track effectiveness.
POST	/chat/checkin	Yes	Saves daily self-assessments (stress level & academic focus on a 1-10 scale).
GET	/analytics/emotions	Yes	Aggregates historical emotion counts for the current user to display in a dashboard pie chart.
GET	/analytics/trend	Yes	Retrieves daily chat frequency volumes to visualize user activity on a dashboard line chart.
GET	/analytics/effectiveness	Yes	Calculates the average user feedback rating to measure overall assistant utility score.
GET	/analytics/stress_academic	Yes	Pulls longitudinal history of stress versus study focus for student tracking.

Section 4: High-Level Design (HLD) Entities

The system follows a classic multi-tier architecture designed to enforce clean separation of responsibilities. The client layer (Angular frontend) communicates entirely via HTTP requests with the FastAPI server. The server is divided into a routing controller tier, a business services tier, an ML/NLP intelligence tier, and a repository storage tier accessing a PostgreSQL database. Below is the relational schema entity layout:

Database Entities & Relations

1. User Entity (Table: users):

- id: Integer (Primary Key, unique auto-increment identifier)
- username: String (Unique, indexed, non-null registration name)
- email: String (Unique, indexed, non-null email identifier)
- password_hash: String (Secure PBKDF2-SHA256 hashed passphrase)
- created_at: DateTime (UTC registration timestamp)

2. ChatLog Entity (Table: chat_logs):
- id: Integer (Primary Key, unique auto-increment identifier)
- user_id: String (Foreign Key linking this chat record to a specific user)
- message: String (The student's incoming message text)
- emotion: String (Detected emotion: sadness, joy, anxiety, stress, or neutral)
- response: String (The synthesized AI assistant output response)
- feedback_score: Integer (Optional student effectiveness rating, scale of 1-5)
- created_at: DateTime (UTC message exchange timestamp)

3. DailyCheckin Entity (Table: daily_checkins):
- id: Integer (Primary Key, unique auto-increment identifier)
- user_id: String (Foreign key linking this check-in to a specific user)
- stress_level: Integer (Subjective stress rating on a scale of 1-10)
- academic_focus: Integer (Subjective academic productivity rating on a scale of 1-10)
- created_at: DateTime (UTC check-in timestamp)

Section 5: Low-Level Design & Code Walkthrough

This section analyzes the actual implementation of the application. It lists the raw source code files directly from the repository, followed by detailed explanations of the design decisions, algorithms, and patterns applied.

5.1 FastAPI Main Entry Point (main.py)

Live Snapshot: backend/main.py

```
import os
from contextlib import asynccontextmanager
from fastapi import FastAPI, Request
from fastapi.responses import FileResponse, HTMLResponse
from dotenv import load_dotenv
from fastapi.middleware.cors import CORSMiddleware

load_dotenv()

from routes.chat_routes import router as chat_router
from routes.auth_routes import router as auth_router
from db.database import Base, engine, wait_for_database

from routes.analytics_routes import router as analytics_router


@asynccontextmanager
async def lifespan(app: FastAPI):
    # Skip long DB connection loops on Vercel serverless environment
    if not os.getenv("VERCEL"):
        wait_for_database()
    try:
        Base.metadata.create_all(bind=engine)
    except Exception as e:
        print(f"Lifespan database creation failed: {e}")
    yield




def create_app() -> FastAPI:
    api = FastAPI(title="AI Mental Health Assistant", lifespan=lifespan)

    @api.get("/health")
    @api.get("/api/health")
    def health():
        from sqlalchemy import text
        db_status = "unknown"
        db_error = None
        try:
            with engine.connect() as connection:
                connection.execute(text("SELECT 1"))
            db_status = "connected"
        except Exception as e:
```

```
            db_status = "failed"
            db_error = str(e)

        return {
            "status": "healthy",
            "database": db_status,
            "database_error": db_error,
            "database_url_configured": bool(os.getenv("DATABASE_URL")),
            "vercel": bool(os.getenv("VERCEL")),
            "openrouter_configured": bool(os.getenv("OPENROUTER_API_KEY")),
        }

    api.add_middleware(
        CORSMiddleware,
        allow_origins=[
            "http://localhost:4200",
            "http://localhost:8000",
            "http://health.34.30.233.97.sslip.io",
            "https://health.34.30.233.97.sslip.io",
            "https://capstone-mental-health.web.app",
            "https://ai-mental-health.blackocean-872335af.centralindia.azure
              containerapps.io",
            "https://virtualgyans.tech",
            "https://www.virtualgyans.tech",
            "https://healthai.virtualgyans.tech",
            "https://virtualgyans.me",
            "http://virtualgyans.me",
            "https://mdirfancse2023.github.io",
            "http://mdirfancse2023.github.io",
        ],
        allow_credentials=True,
        allow_methods=["*"],
        allow_headers=["*"],
    )

    api.include_router(auth_router)
    api.include_router(auth_router, prefix="/api")
    api.include_router(chat_router)
    api.include_router(chat_router, prefix="/api")
    api.include_router(analytics_router)
    api.include_router(analytics_router, prefix="/api")

    @api.get("/")
    def read_root():
        return {
            "message": "AI Powered Mental Health Assistant API is running",
            "docs": "/docs",
            "health": "/health"
        }

    return api


app = FastAPI()
# Serve at both root and /aimentalhealth for compatibility
app.mount("", create_app())
app.mount("/aimentalhealth", create_app())
```

Explanation of main.py:

1. Decorators & Context Managers: It utilizes an asynchronous context manager (@asynccontextmanager) named lifespan to manage the server lifecycle. On startup, it triggers database creation (Base.metadata.create_all). It checks for a VERCEL environment variable to skip long database connection loops in serverless hosting.
2. Database Wait Logic: It executes wait_for_database() on local dev to ensure PostgreSQL is active, preventing race conditions in containerized environments.
3. Middleware Integration: CORSMiddleware is added to support safe cross-origin requests from Angular (running on localhost:4200, GitHub Pages, or Azure Container Apps).
4. Multi-Mount Configuration: The application mounts the API router at both the root level and under a prefix (/aimentalhealth) to ensure backwards compatibility with previous production deployment versions.

5.2 Database & Session Settings (db/database.py)

Live Snapshot: backend/db/database.py

```
import os
import time
from sqlalchemy import create_engine, text
from sqlalchemy.exc import OperationalError
from sqlalchemy.orm import sessionmaker, declarative_base

DATABASE_URL = os.getenv("DATABASE_URL")
if DATABASE_URL:
    if DATABASE_URL.startswith("postgres://"):
        DATABASE_URL = DATABASE_URL.replace("postgres://", "postgresql://", 1)
else:
    # Fallback to in-memory SQLite to prevent startup crashes when env vars
      are missing
    DATABASE_URL = "sqlite:///:memory:"

print("DB URL:", DATABASE_URL)  # optional debug

engine = create_engine(DATABASE_URL, echo=True, pool_pre_ping=True)

SessionLocal = sessionmaker(bind=engine)

Base = declarative_base()

def wait_for_database(max_attempts: int = 30, delay_seconds: int = 2) -> None:
    last_error = None

    for attempt in range(1, max_attempts + 1):
        try:
            with engine.connect() as connection:
                connection.execute(text("SELECT 1"))
            print("Database connection established.")
            return
        except OperationalError as exc:
            last_error = exc
            print(f"Database not ready (attempt {attempt}/{max_attempts}): {
              exc}")
            if attempt < max_attempts:
                time.sleep(delay_seconds)

    raise last_error
```

Explanation of database.py:

1. Dynamic Connection Routing: The script inspects the DATABASE_URL environment variable. In serverless hosting like Vercel, PostgreSQL connection formats sometimes use a legacy 'postgres://' scheme, which SQLAlchemy no longer supports natively. The script automatically replaces this prefix with 'postgresql://'.

2. Safe SQLite Fallback: If no DATABASE_URL is configured (for example, in a local testing environment), the script automatically falls back to an in-memory SQLite database ('sqlite:///:memory:'), allowing the API to startup cleanly without hard crashing.

3. Connection Resiliency: The wait_for_database() function runs a retry loop (up to 30 attempts, sleeping for 2 seconds between retries) checking database connectivity with 'SELECT 1'. This is crucial in docker-compose deployments, as the database container takes several seconds to boot and accept TCP socket connections.

5.3 User Database Model (models/user_model.py)

Live Snapshot: backend/models/user_model.py

```
from datetime import datetime

from sqlalchemy import Column, DateTime, Integer, String

from db.database import Base


class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True)
    username = Column(String, unique=True, index=True, nullable=False)
    email = Column(String, unique=True, index=True, nullable=False)
    password_hash = Column(String, nullable=False)
    created_at = Column(DateTime, default=datetime.utcnow, nullable=False)
```

Explanation of user_model.py:

1. Base Class Inheritance: Inherits from the SQLAlchemy declarative Base class, linking it to the metadata engine.

2. Table Definition: Maps the class to the 'users' table in PostgreSQL. Declares fields with indexing (index=True) and uniqueness constraints (unique=True) for usernames and emails to optimize queries.

3. Default Timestamps: Utilizes datetime.utcnow (passed as a callable reference, without parentheses, so it executes exactly at the moment a new record is created) to assign default registration times.

5.4 ChatLog & DailyCheckin Models (models/chat_model.py)

Live Snapshot: backend/models/chat_model.py

```
from sqlalchemy import Column, Integer, String, DateTime
from datetime import datetime
from db.database import Base

class ChatLog(Base):
    __tablename__ = "chat_logs"

    id = Column(Integer, primary_key=True, index=True)
    user_id = Column(String, default="default_user")   # [Alert] NEW
    message = Column(String)
    emotion = Column(String)
    response = Column(String)
    feedback_score = Column(Integer, default=0) # 1 to 5 scale, 0 means unrated
    created_at = Column(DateTime, default=datetime.utcnow)

class DailyCheckin(Base):
    __tablename__ = "daily_checkins"

    id = Column(Integer, primary_key=True, index=True)
    user_id = Column(String, default="default_user")
    stress_level = Column(Integer)  # 1 to 10
    academic_focus = Column(Integer) # 1 to 10
```

```
    created_at = Column(DateTime, default=datetime.utcnow)
```

Explanation of chat_model.py:

1. ChatLog Structure: Stores conversation states. The user_id field is defined as a String (default 'default_user') to link conversations to user profiles. It contains fields storing the input message, detected emotion, and generated response.

2. Feedback Mechanism: Includes a feedback_score field (Integer, default 0, meaning unrated). This field is updated when students submit a quality rating (1-5), allowing the system to log AI effectiveness.

3. DailyCheckin: Stores self-reported numerical stress and academic focus scales. These two integers represent longitudinal trends to track correlation (e.g., if high academic focus reduces stress over time).

5.5 Request Validation Schemas (models/request_models.py)

Live Snapshot: backend/models/request_models.py

```
from pydantic import BaseModel


class ChatRequest(BaseModel):
    message: str
    user_id: str = "default_user"


class FeedbackRequest(BaseModel):
    score: int  # e.g., 1 to 5, or -1, 1


class DailyCheckinRequest(BaseModel):
    user_id: str = "default_user"
    stress_level: int
    academic_focus: int


class SignupRequest(BaseModel):
    username: str
    email: str
    password: str


class LoginRequest(BaseModel):
    identifier: str
    password: str
```

Explanation of request_models.py:

1. Pydantic Type Enforcement: Extends Pydantic's BaseModel to enforce strict type checking on incoming REST requests. If a client submits a string for a field expecting an integer, FastAPI automatically returns an HTTP 422 Unprocessable Entity.

2. Signup/Login Schemas: SignupRequest forces username, email, and password, whereas LoginRequest accepts a flexible identifier field (allowing students to sign in using either their username or their email address) alongside password.

3. Default fallbacks: ChatRequest and DailyCheckinRequest assign a default user_id of 'default_user' to maintain compatibility if an unauthenticated endpoint is used.

5.6 Cryptographic Authentication & JWT Service (services/auth_service.py)

Live Snapshot: backend/services/auth_service.py

```
import base64
import hashlib
import hmac
import json
import os
import secrets
import time

from fastapi import Header, HTTPException, status

from db.user_repository import (
    create_user,
    get_user_by_email,
    get_user_by_id,
    get_user_by_identifier,
    get_user_by_username,
)
from models.user_model import User

AUTH_SECRET_KEY = os.getenv("AUTH_SECRET_KEY", "mental-health-dev-secret")
TOKEN_TTL_SECONDS = int(os.getenv("AUTH_TOKEN_TTL_SECONDS", "604800"))
PASSWORD_ITERATIONS = 390000


def _base64url_encode(value: bytes) -> str:
    return base64.urlsafe_b64encode(value).rstrip(b"=").decode("ascii")


def _base64url_decode(value: str) -> bytes:
    padding = "=" * (-len(value) % 4)
    return base64.urlsafe_b64decode(value + padding)


def _bad_auth_error(detail: str = "Invalid authentication credentials") -> H
  TTPException:
    return HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail=detail,
        headers={"WWW-Authenticate": "Bearer"},
    )


def serialize_user(user: User) -> dict[str, str | int]:
    return {
        "id": user.id,
        "username": user.username,
        "email": user.email,
    }


def hash_password(password: str) -> str:
    salt = secrets.token_bytes(16)
    digest = hashlib.pbkdf2_hmac("sha256", password.encode("utf-8"), salt, P
```

```
      ASSWORD_ITERATIONS)
    return (
        f"pbkdf2_sha256${PASSWORD_ITERATIONS}$"
        f"{_base64url_encode(salt)}${_base64url_encode(digest)}"
    )


def verify_password(password: str, password_hash: str) -> bool:
    try:
        algorithm, iterations, salt_value, digest_value = password_hash.spli
          t("$", 3)
        if algorithm != "pbkdf2_sha256":
            return False
        salt = _base64url_decode(salt_value)
        expected_digest = _base64url_decode(digest_value)
        actual_digest = hashlib.pbkdf2_hmac(
            "sha256",
            password.encode("utf-8"),
            salt,
            int(iterations),
        )
        return hmac.compare_digest(actual_digest, expected_digest)
    except (ValueError, TypeError):
        return False


def create_access_token(user: User) -> str:
    header = {"alg": "HS256", "typ": "JWT"}
    payload = {
        "sub": str(user.id),
        "username": user.username,
        "email": user.email,
        "exp": int(time.time()) + TOKEN_TTL_SECONDS,
    }

    encoded_header = _base64url_encode(json.dumps(header, separators=(",", "
      :")).encode("utf-8"))
    encoded_payload = _base64url_encode(json.dumps(payload, separators=(",",
       ":")).encode("utf-8"))
    unsigned_token = f"{encoded_header}.{encoded_payload}"
    signature = hmac.new(
        AUTH_SECRET_KEY.encode("utf-8"),
        unsigned_token.encode("utf-8"),
        hashlib.sha256,
    ).digest()
    return f"{unsigned_token}.{_base64url_encode(signature)}"


def decode_access_token(token: str) -> dict[str, str | int]:
    try:
        encoded_header, encoded_payload, encoded_signature = token.split(".", 2)
    except ValueError as exc:
        raise _bad_auth_error() from exc

    unsigned_token = f"{encoded_header}.{encoded_payload}"
    expected_signature = hmac.new(
        AUTH_SECRET_KEY.encode("utf-8"),
```

```
        unsigned_token.encode("utf-8"),
        hashlib.sha256,
    ).digest()

    if not hmac.compare_digest(_base64url_encode(expected_signature), encode
      d_signature):
        raise _bad_auth_error()

    try:
        payload = json.loads(_base64url_decode(encoded_payload).decode("utf-8"))
    except (json.JSONDecodeError, UnicodeDecodeError) as exc:
        raise _bad_auth_error() from exc

    if int(payload.get("exp", 0)) <= int(time.time()):
        raise _bad_auth_error("Token has expired")

    return payload


def _token_response(user: User) -> dict[str, str | dict[str, str | int]]:
    return {
        "access_token": create_access_token(user),
        "token_type": "bearer",
        "user": serialize_user(user),
    }


def register_user(username: str, email: str, password: str) -> dict[str, str
   | dict[str, str | int]]:
    normalized_username = username.strip()
    normalized_email = email.strip().lower()

    if len(normalized_username) < 3:
        raise ValueError("Username must be at least 3 characters long.")
    if "@" not in normalized_email or "." not in normalized_email:
        raise ValueError("Please provide a valid email address.")
    if len(password) < 8:
        raise ValueError("Password must be at least 8 characters long.")
    if get_user_by_username(normalized_username):
        raise ValueError("That username is already in use.")
    if get_user_by_email(normalized_email):
        raise ValueError("That email is already in use.")

    user = create_user(normalized_username, normalized_email, hash_password(
      password))
    return _token_response(user)


def login_user(identifier: str, password: str) -> dict[str, str | dict[str,
  str | int]]:
    normalized_identifier = identifier.strip().lower()
    user = get_user_by_identifier(normalized_identifier)

    if not user:
        user = get_user_by_identifier(identifier.strip())

    if not user or not verify_password(password, user.password_hash):
```

```
        raise _bad_auth_error("Invalid username/email or password.")

    return _token_response(user)


def get_current_user(authorization: str | None = Header(default=None)) -> User:
    if not authorization or not authorization.startswith("Bearer "):
        raise _bad_auth_error("Authentication required.")

    token = authorization.split(" ", 1)[1].strip()
    payload = decode_access_token(token)

    try:
        user_id = int(payload["sub"])
    except (KeyError, TypeError, ValueError) as exc:
        raise _bad_auth_error() from exc

    user = get_user_by_id(user_id)
    if not user:
        raise _bad_auth_error("User no longer exists.")

    return user
```

Explanation of auth_service.py:

1. Password Hashing (PBKDF2-HMAC-SHA256): Instead of vulnerable custom hashing, the auth service uses the standard hashlib.pbkdf2_hmac with a high iteration count (390,000 iterations). It generates a random 16-byte salt, hashes the password, and encodes both the salt and the digest using URL-safe Base64 encoding. The resulting hash format (pbkdf2_sha256$390000$salt$digest) matches industry standards.

2. Stateful Password Verification: Uses hmac.compare_digest() for password verification. This constant-time comparison is critical to block timing-based side-channel attacks.

3. Custom JWT Token Generation: The create_access_token() function manually constructs a JSON Web Token. It encodes the JWT header and payload as JSON, base64url-encodes them, joins them with a dot, signs the unsigned token string using hmac-sha256 with the AUTH_SECRET_KEY, and appends the signature. This implements JWT token generation without third-party dependencies.

4. Dependency Injection: The get_current_user() function acts as a FastAPI dependency. It extracts the Authorization header, splits the bearer token, verifies its signature, checks the expiration timestamp (exp), and fetches the user.

5.7 Chat Stream & Context Pipeline Service (services/chat_service.py)

Live Snapshot: backend/services/chat_service.py

```
import json
from db.chat_repository import save_chat, get_recent_chats, get_emotion_summary
from ml.emotion_model import detect_emotion
from ml.rag_model import retrieve_context
from ml.recommendation import generate_recommendation
from services.llm_service import generate_llm_stream

def process_chat_stream(message: str, user_id="default_user"):

    # [Brain] Emotion detection
    emotion = detect_emotion(message)

    # [Therapy] Clinical Recommendation based solely on emotion class
    therapy_recommendation = generate_recommendation(emotion)

    # [RAG] RAG Context Retrieval (VECTOR SEARCH)
    rag_context = retrieve_context(message)

    # [Brain] MEMORY: get last chats
    past_chats = get_recent_chats(user_id)

    context = ""
    for chat in reversed(past_chats):
        context += f"User: {chat.message}\nAssistant: {chat.response}\n"

    # [Analytics] PERSONALIZATION: emotion trend
    emotion_summary = get_emotion_summary(user_id)

    trend = ", ".join([f"{k}:{v}" for k, v in emotion_summary.items()])

    # [Brain] ELITE PROMPT ENGINEERING
    prompt = f"""
You are an elite, highly advanced AI Clinical Mental Health Assistant design
  ed specifically to help university students. Your goal is to sound as intell
  igent, articulate, and empathetic as a Master's-level therapist or high-end
  life coach (like ChatGPT-4).

You must organically synthesize the following backend data streams constrain
  t into your response smoothly:
- User's Current Emotion Vector: {emotion}
- User's Historical Emotion Trend: {trend}
- Required [Clinical Therapy Recommendation]: {therapy_recommendation}
{rag_context}

Conversation History Context:
{context}

Current Student Message: {message}

OUTPUT RULES (CRITICAL):
1. Give a deeply nuanced, empathetic, and highly intelligent psychological r
  esponse. DO NOT sound like a generic robot.
2. Validate their feelings deeply utilizing cognitive behavioral therapy (CB
```

```
  T) conversational principles.
3. If [Retrieved University Mental Health Resources] or [Clinical Therapy] a
  re provided above, explicitly integrate them flawlessly into your reasoning.
   Explain *why* these mechanisms biologically or psychologically work for the
  ir specific emotion.
4. Structure your response elegantly like an advanced AI: heavily use native
   Markdown, bolding key insights, breaking long text into very readable logic
  al bullet points, and using relevant emojis to maintain warmth.
5. Provide immediately actionable, step-by-step guidance.
"""

    #  LLM streaming call
    full_response = ""
    for chunk in generate_llm_stream(prompt, emotion):
        full_response += chunk
        # Yield Server-Sent Event formatted chunks
        yield f"data: {json.dumps({'chunk': chunk})}\n\n"

    #  Save full completed chat to database
    chat_id = save_chat(user_id, message, emotion, full_response)

    # Yield final metadata so frontend can hook up feedback buttons
    yield f"data: {json.dumps({'chat_id': chat_id, 'emotion': emotion})}\n\n"

    # Indicate end of stream
    yield "data: [DONE]\n\n"
```

Explanation of chat_service.py:

1. Orchestration Flow: This file binds the entire assistant pipeline. For every user chat request, it runs emotion detection, maps it to a clinical coping strategy, queries the RAG system for university resources, pulls conversation history context, and queries the user's historical emotional trends.

2. Prompt Engineering: Constructs a detailed clinical prompt. It instructs the LLM to behave as a Master's-level therapist, validate student feelings using Cognitive Behavioral Therapy (CBT) conversational patterns, explain the physiological/psychological reasons why the coping mechanisms work, structure output with native markdown (bullet points, bold text, emojis), and provide immediate, actionable instructions.

3. Generator & SSE Yielding: Yields Server-Sent Events (SSE) data stream chunks as they arrive from OpenRouter. After the stream completes, it calls save_chat() to persist the conversation logs and yields the database ID and detected emotion to the frontend for feedback ratings.

5.8 OpenRouter API Stream Handler (services/llm_service.py)

Live Snapshot: backend/services/llm_service.py

```
import os
import requests

OPENROUTER_API_KEY = os.getenv("OPENROUTER_API_KEY")

import json

def generate_llm_stream(prompt: str, emotion: str = ""):
    url = "https://openrouter.ai/api/v1/chat/completions"

    headers = {
        "Authorization": f"Bearer {OPENROUTER_API_KEY}",
        "Content-Type": "application/json"
    }

    data = {
        "model": "openai/gpt-3.5-turbo",
        "stream": True,
        "messages": [
            {
                "role": "user",
                "content": prompt   # [Alert] FULL CONTEXT PROMPT
            }
        ]
    }

    try:
        response = requests.post(url, headers=headers, json=data, stream=True)

        if response.status_code != 200:
            print("STATUS:", response.status_code, response.text)
            yield "I'm here for you. Please tell me more."
            return

        for line in response.iter_lines():
            if line:
                decoded_line = line.decode('utf-8')
                if decoded_line.startswith("data: "):
                    json_str = decoded_line[6:]
                    if json_str.strip() == "[DONE]":
                        break
                    try:
                        chunk_data = json.loads(json_str)
                        if "choices" in chunk_data and len(chunk_data["choic
                          es"]) > 0:
                            delta = chunk_data["choices"][0].get("delta", {})
                            if "content" in delta:
                                yield delta["content"]
                    except Exception as e:
                        print("Stream parse err:", e)
                        pass

    except Exception as e:
```

```
        print("LLM ERROR:", e)
        yield "I'm here for you. Please tell me more."
```

Explanation of llm_service.py:

1. Streaming POST Requests: Utilizes the requests library with stream=True. This maintains an open HTTP connection allowing chunked response bodies to be processed asynchronously.

2. SSE Parsing Loop: Iterates over the raw bytes of the response using response.iter_lines(). It ignores empty keep-alive lines, decodes the lines into UTF-8, strips the 'data: ' prefix, and loads the payload using json.loads.

3. Content Delta Extraction: Traverses the parsed dictionary choices -> delta -> content. Yields each token to the calling generator immediately. If it encounters the standard SSE '[DONE]' flag, it terminates the parsing loop.

4. Error Resiliency: Contains a try/except wrap. If OpenRouter returns an error code (such as quota exceeded) or encounters a network timeout, it yields a safe, empathetic fallback message to avoid breaking the frontend chat UI.

5.9 LLM & Heuristic Emotion Detection (ml/emotion_model.py)

Live Snapshot: backend/ml/emotion_model.py

```
import os
import requests

OPENROUTER_API_KEY = os.getenv("OPENROUTER_API_KEY")

def detect_emotion(text: str) -> str:
    """Detect emotion using OpenRouter API"""
    try:
        response = requests.post(
            "https://openrouter.ai/api/v1/chat/completions",
            headers={
                "Authorization": f"Bearer {OPENROUTER_API_KEY}",
                "Content-Type": "application/json"
            },
            json={
                "model": "openai/gpt-3.5-turbo",
                "messages": [
                    {
                        "role": "user",
                        "content": f"""Analyze the emotion in this text and
                          respond with ONLY one word from: happy, sad, anxiety
                          , stress, or neutral.

Text: "{text}"

Response:"""
                    }
                ],
                "temperature": 0.3,
                "max_tokens": 10
            }
        )

        if response.status_code == 200:
            emotion = response.json()["choices"][0]["message"]["content"].st
              rip().lower()
            # Validate and map emotions
            valid_emotions = ["happy", "sad", "anxiety", "stress", "neutral"]
            return emotion if emotion in valid_emotions else "neutral"
        else:
            print(f"Emotion detection API error: {response.status_code}")
            return "neutral"

    except Exception as e:
        print(f"Emotion detection error: {e}")
        # Fallback to simple heuristics if API fails
        text = text.lower()
        if "stress" in text or "pressure" in text: return "stress"
        if "sad" in text or "depressed" in text: return "sad"
        if "happy" in text or "great" in text: return "happy"
        if "anxious" in text or "worry" in text: return "anxiety"
        return "neutral"
```

Explanation of emotion_model.py:

1. Structured Extraction: Queries OpenRouter to categorize the text emotion. Instructs the LLM to return exactly one word from: happy, sad, anxiety, stress, or neutral, setting the temperature to 0.3 and max_tokens to 10 for deterministic classification.
2. Regex Fallback: In cases of network outages or API failures, a rule-based regex fallback matches key words (e.g., 'pressure' maps to stress, 'depressed' maps to sad, 'worry' maps to anxiety) to ensure the system is resilient.

5.10 Clinical Recommendation Matrix (ml/recommendation.py)

Live Snapshot: backend/ml/recommendation.py

```
import random

# A structured, clinically grounded recommendation matrix mapped to the exact
# 6 emotional classes natively predicted by Hugging Face's 'distilbert-base-
  uncased-emotion'.
CLINICAL_RECOMMENDATION_MATRIX = {
    "sadness": [
        "Behavioral Activation: Start by setting one micro-goal today, like
          organizing your desk or taking a 10-minute walk outside to trigger a
           baseline dopamine release.",
        "Cognitive Reframing: Write down three negative thoughts you had tod
          ay, and objectively challenge their factual accuracy on paper.",
        "Social Grounding: Reach out to a trusted peer or mentor, even just
          via text, to break the loop of academic isolation."
    ],
    "joy": [
        "Momentum Building: You are in an excellent cognitive state! Capital
          ize on this by tackling your hardest academic challenge today.",
        "Gratitude Journaling: Solidify positive neural pathways by writing
          down two specific things that brought you joy today.",
        "Social Reinforcement: Share your success or positive energy with a
          friend to reinforce communal bonding."
    ],
    "love": [
        "Connection Maintenance: Empathy and connection are high. Dedicate 2
          0 minutes today to strictly non-academic socializing.",
        "Self-Compassion: Direct that empathy inward. Acknowledge your hard
          work and allow yourself a guilt-free evening of rest."
    ],
    "anger": [
        "Physiological De-escalation: Try the 'Mammalian Dive Reflex'. Splas
          h freezing water on your face to forcibly slow your autonomic heart
          rate.",
        "Constructive Venting: Do a 'brain dump'. Write furiously on a piece
           of paper for 5 minutes, then literally tear it up to release kineti
          c frustration.",
        "Distance the Stressor: If you are angry at a specific assignment or
           grade, immediately step away. Do not email a professor or peer whil
          e in a heightened emotional state."
    ],
    "fear": [
        "Autonomic Regulation: Use Box Breathing (Inhale 4s, Hold 4s, Exhale
           4s, Hold 4s) to directly combat the 'Fight or Flight' sympathetic n
          ervous response.",
```

```
        "The 5-4-3-2-1 Grounding Method: Identify 5 things you see, 4 you ca
          n touch, 3 you hear, 2 you smell, and 1 you taste. This prevents anx
          iety spiraling.",
        "Probability Mapping: Ask yourself: 'What is the absolute worst case
           scenario?' Usually, explicitly naming the fear removes its psycholo
          gical power."
    ],
    "surprise": [
        "Cognitive Processing: Unexpected events spike cortisol. Take 5 minu
          tes to sit quietly and process how this new information changes your
           current academic plan.",
        "Adaptive Re-planning: If the surprise was negative (like an unexpec
          ted quiz), don't panic. Quickly sketch out a new 3-step recovery tim
          eline."
    ]
}

def generate_recommendation(emotion: str) -> str:
    """
    Dynamically recommends an empirically-backed psychological coping mechanism
    based on the localized Hugging Face Emotion classification.
    """
    normalized_emotion = emotion.lower().strip()

    if normalized_emotion in CLINICAL_RECOMMENDATION_MATRIX:
        # Randomly select a strategy from the specific emotional bucket to p
          revent conversational fatigue
        strategies = CLINICAL_RECOMMENDATION_MATRIX[normalized_emotion]
        return random.choice(strategies)

    return "I'm always here for you. Continuing to talk out your feelings wi
      th me is the best first step!"
```

Explanation of recommendation.py:

1. Clinically Grounded Mapping: Maps predicted emotional classes directly to empirically backed psychological coping mechanisms. The mapping covers sadness (CBT activation), anger (physiological de-escalation via dive reflex), fear (autonomic box breathing and grounding), and surprise (cognitive adaptive planning).
2. Random Selection: Each emotional bucket has multiple strategies. The system calls random.choice() to select a strategy, preventing conversational fatigue.

5.11 Custom Vector RAG Retrieval Engine (ml/rag_model.py)

Live Snapshot: backend/ml/rag_model.py

```
import os
import json
import requests

OPENROUTER_API_KEY = os.getenv("OPENROUTER_API_KEY")
RESOURCE_FILE = os.path.join(os.path.dirname(os.path.dirname(__file__)), "da
  ta", "resources.json")

knowledge_base = []

def load_knowledge_base():
    """Load the JSON resources"""
    global knowledge_base

    if not os.path.exists(RESOURCE_FILE):
        print(f"RAG WARNING: Knowledge base not found at {RESOURCE_FILE}")
        return

    with open(RESOURCE_FILE, "r") as f:
        knowledge_base = json.load(f)
    print(f"RAG INIT: Loaded {len(knowledge_base)} resources")

def retrieve_context(query: str, top_k: int = 1) -> str:
    """
    Use OpenRouter API to find the most relevant resources from knowledge base
    """
    if not knowledge_base:
        return ""

    try:
        # Create a prompt to find the most relevant resource
        resources_text = "\n\n".join([
            f"Resource {i+1}: {item.get('title', 'Unknown')}\nContent: {item
              ['content'][:200]}..."
            for i, item in enumerate(knowledge_base[:5])  # Limit to top 5 t
              o keep context small
        ])

        response = requests.post(
            "https://openrouter.ai/api/v1/chat/completions",
            headers={
                "Authorization": f"Bearer {OPENROUTER_API_KEY}",
                "Content-Type": "application/json"
            },
            json={
                "model": "openai/gpt-3.5-turbo",
                "messages": [
                    {
                        "role": "user",
                        "content": f"""Given these university mental health
                          resources:

{resources_text}
```

```
User query: "{query}"

Return ONLY the number (1-5) of the most relevant resource. Just the number,
   nothing else."""
                    }
                ],
                "temperature": 0.1,
                "max_tokens": 5
            }
        )

        if response.status_code == 200:
            try:
                result = response.json()["choices"][0]["message"]["content"]
                  .strip()
                idx = int(result) - 1
                if 0 <= idx < len(knowledge_base):
                    content = knowledge_base[idx]["content"]
                    return f"[Retrieved Mental Health Resource: {knowledge_b
                      ase[idx].get('title', 'Resource')}]\n{content}"
            except (ValueError, IndexError, KeyError):
                pass

        return ""

    except Exception as e:
        print(f"RAG retrieval error: {e}")
        return ""

# Initialize knowledge base immediately when this module is imported by FastAPI
load_knowledge_base()
```

Explanation of rag_model.py:

1. Dynamic Content Fetching: RAG context retrieval load_knowledge_base() loads the academic coping resources JSON file upon importing the backend module.

2. Semantic Resource Router: The retrieve_context() function implements a semantic matching engine. It constructs a consolidated prompt containing academic resource details alongside the user query. It calls OpenRouter with temperature 0.1 (low randomness) and queries GPT-3.5 to return ONLY the index of the most relevant resource. It parses this integer index and returns the title and content of the matching mechanism, injecting empirical university support directly into the therapist prompt.

5.12 Analytics & Dashboard Statistics (services/analytics_service.py)

Live Snapshot: backend/services/analytics_service.py

```
from db.database import SessionLocal
from models.chat_model import ChatLog
from collections import Counter


def get_emotion_distribution(user_id: str):
    db = SessionLocal()
    try:
        chats = db.query(ChatLog)\
            .filter(ChatLog.user_id == user_id)\
            .all()

        emotions = [chat.emotion for chat in chats]
        result = dict(Counter(emotions))
        return result
    finally:
        db.close()


def get_chat_trend(user_id: str):
    db = SessionLocal()
    try:
        chats = db.query(ChatLog)\
            .filter(ChatLog.user_id == user_id)\
            .all()

        trend = {}

        for chat in chats:
            date = chat.created_at.strftime("%Y-%m-%d")
            trend[date] = trend.get(date, 0) + 1

        return trend
    finally:
        db.close()

from models.chat_model import DailyCheckin


def get_effectiveness_score(user_id: str):
    db = SessionLocal()
    try:
        chats = db.query(ChatLog).filter(ChatLog.user_id == user_id, ChatLog
          .feedback_score != 0).all()

        if not chats:
            return 0.0

        total_score = sum(chat.feedback_score for chat in chats)
        avg_score = total_score / len(chats)
        return round(avg_score, 2)
    finally:
        db.close()
```

```
def get_stress_and_academic_trend(user_id: str):
    db = SessionLocal()
    try:
        checkins = db.query(DailyCheckin).filter(DailyCheckin.user_id == use
          r_id).order_by(DailyCheckin.created_at.asc()).all()

        trend = []
        for checkin in checkins:
            trend.append({
                "date": checkin.created_at.strftime("%Y-%m-%d"),
                "stress_level": checkin.stress_level,
                "academic_focus": checkin.academic_focus
            })

        return trend
    finally:
        db.close()
```

Explanation of analytics_service.py:
1. Counter Utility: The get_emotion_distribution() function fetches all chat logs for a given user, extracts the emotion field, and uses Python's collections.Counter to construct a frequency distribution dictionary, which is converted into a chart-ready format for the frontend Angular dashboard.
2. Temporal Tracking: The get_chat_trend() function parses the created_at timestamp of user chats and groups them by date string (YYYY-MM-DD), plotting daily conversation volume trends.
3. System Effectiveness: The get_effectiveness_score() calculates the running average of user feedback scores. It filters out unrated records (feedback_score == 0) and computes a neat floating average rounded to 2 decimal places.
4. Correlation Trends: The get_stress_and_academic_trend() function fetches user self-assessments over time, sorting them chronologically, returning stress levels alongside academic productivity to evaluate behavioral patterns.

Section 6: Summary & System Verification

The system has been thoroughly verified using localized developer server testing, integration test configurations, and automated pipeline scans. The static analysis is run automatically via JetBrains Qodana backend pipelines, while deployment pathways run through GitHub Actions (publishing the Angular frontend to GitHub Pages) and Vercel (hosting the FastAPI Python server with serverless database endpoints).

In conclusion, this Capstone MBA Data Science project successfully demonstrates how modern AI/NLP tools can be integrated into responsive, secure web platforms to solve high-impact, real-world problems like student mental health.